Codepup

Claude Skills Hands-On Session

Rajesh Pentakota

Founder, CodePup AI (https://www.linkedin.com/in/rajeshpeko/)

Ex-Product Manager at Zynga, Flipkart, Walmart

SESSION OVERVIEW

Today's Agenda

1

PART 1

Claude Skills 101

What skills are, how they work, and why they matter for PMs & Founders

2

DEMO 1

Feature PRD Skill

Turn a vague feature idea into an implementation-ready PRD in one command

3

DEMO 2

Blog Writer Skill

Generate a full blog post with structured output and image script from a single idea

Codepup

Why Skills Matter

The Problem with Normal Prompting

- You repeat the same instructions every single session
- Quality depends entirely on how well you prompt each time
- Complex workflows require multiple steps, files, and context-setting
- PM and founder workflows need **consistency**, not creativity

What Skills Solve

→ **Reusable instructions**

Define once, invoke anywhere

→ **Better consistency**

Same structure, every time

→ **Less token waste**

No more re-explaining context

→ **Easier team handoff**

Share workflows, not just prompts

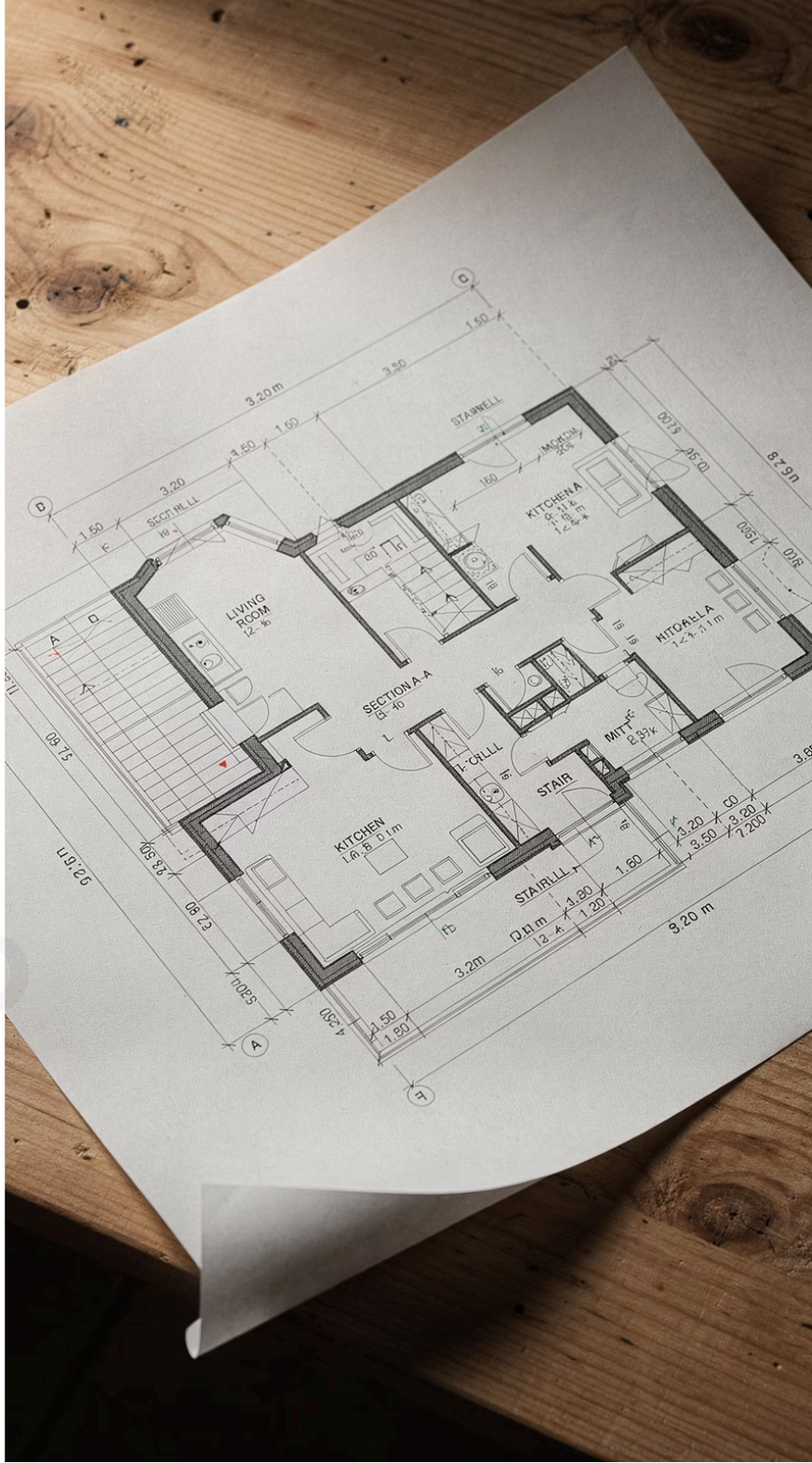
Codepup

What Is a Claude Skill?

A skill is a **reusable capability** Claude can load when needed. Defined using a `SKILL.md` file, it can be invoked directly with a slash command, load automatically when relevant, and include instructions, examples, references, and scripts.

1	**Prompt** The core instructions that define what Claude should do and how
2	**Rules** Frontmatter controls: who invokes it, what tools it can use, how it behaves
3	**Optional Files** Brand guides, examples, and reference docs Claude can draw from
4	**Optional Scripts** Shell scripts that preprocess inputs like PDFs or trigger automation

Simple formula: `Skill = Prompt + Rules + Optional Files + Optional Scripts` — far more powerful than saving a prompt in Notes.



Basic Anatomy of a Skill

Every skill starts with a `SKILL.md` file. The frontmatter defines *how* the skill behaves; the markdown body defines *what* Claude actually does.

The SKILL.md Structure

```
---
name: feature-breakdown
description: Convert vague product ideas
  into a structured PRD with acceptance
  criteria and implementation clarity
---

Turn the given feature request into a
complete PRD. Include:

1. Problem statement
2. User stories
3. Scope
4. Acceptance criteria
5. Edge cases
```

Key Parts Explained

`name`

Becomes the slash command — e.g. `/feature-breakdown`

`description`

Helps Claude understand when to auto-load the skill

Markdown body

The actual workflow instructions Claude follows step by step

Codepup

How Skills Get Invoked & Where They Live

Two Ways to Invoke a Skill

Manual Invocation

Type `/feature-breakdown` or `/blog-writer` — gives you full control. Best for side-effect-heavy workflows like deploy, publish, or send.

Automatic Invocation

Claude detects your intent and loads the right skill automatically. Best for reference or guidance skills where convenience matters more than control.

Where Skills Live

🏠 Personal

`~/.claude/skills/` — available across all your projects. Your reusable toolkit.

📁 Project

`.claude/skills/` — scoped to one codebase. Team and repo conventions.

🏢 Enterprise

Org-wide distribution or bundled with plugins for broader teams.

Codepup

More Than Just a Prompt

A mature skill is a **mini operating system** for your workflow. Supporting files keep the main skill clean while giving Claude rich context to work from.

Brand & Example Files

`brand-language.md` and `examples.md` give Claude a consistent voice and concrete outputs to imitate — dramatically improving quality.

Reference Documentation

`reference.md` stores detailed context you don't want cluttering the main skill file — frameworks, definitions, or domain knowledge.

Preprocessing Scripts

`scripts/pdf-to-md.sh` and `doc-to-md.sh` convert raw source materials before Claude ever sees them — cleaner inputs, better outputs.

DEMO 1

Feature Breakdown Skill

Convert a vague feature idea from a PM or founder into an **implementation-ready PRD** — in one repeatable command.

Skill Output Includes

- Problem statement
- User stories
- Scope and non-scope boundaries
- Acceptance criteria
- Edge cases
- Implementation-ready PRD

What Makes This Skill Strong

- **Brand language file** ensures consistent tone and vocabulary across all PRDs
- **Example outputs** give Claude a quality bar to match rather than invent
- **PDF/DOC converter scripts** post process markdown material automatically
- **Reduces ambiguity** and gives engineering clearer, actionable specs

From vague idea → structured PRD in one repeatable command. This is a high-leverage PM skill that converts fuzzy thinking into structured execution.

NEW COHORT

AI Coding for Product Managers

Build Real Products in 6 Weeks

- 📅 **Cohort Starts:** March 27th
- ⏱️ **Duration:** 6 Weeks
- 👨‍🏫 **Instructor:** Rajesh P
- 🔗 **Platform:** Maven
- PROMO CODE: CODEPUP50

Go from idea to shipped product using AI — no prior coding required.

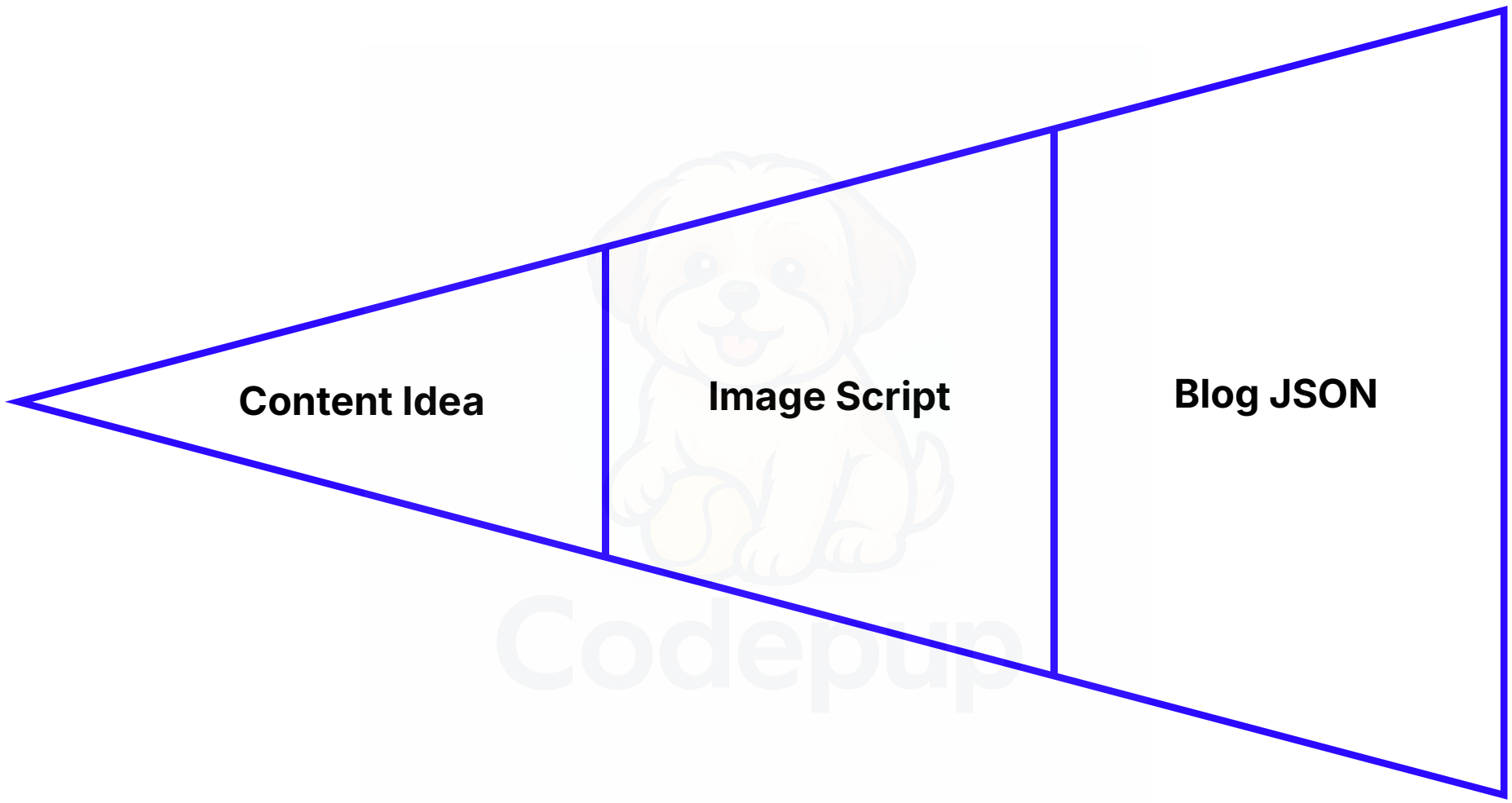
Scan to Join

https://maven.com/rajeshpeko/idea2prod

DEMO 2

Blog Writer Skill

This is not just writing help — it is a **reusable content production workflow** that moves from a content idea all the way to a publishable, live-ready artifact.



What This Skill Showcases

- Skills can **trigger real workflows**, not just produce text responses
- Skills can generate **machine-friendly structured output** like JSON
- Combines writing + structured output + automation in one command

Why It Matters for Founders

- Reduces multi-step manual effort to a single invocation
- Great for marketing, content, and growth use cases
- Massive token savings through standardised workflow design

Frontmatter Controls Behaviour

The frontmatter is the **brain of the skill**. Understanding just a few key fields is enough to build very powerful, well-controlled workflows.

`name` & `description`

Sets the slash command and helps Claude decide when to auto-load the skill

`argument-hint`

Shows users what input to pass when invoking the skill manually

`disable-model-invocation`

Set `true` to ensure only you can trigger it — essential for deploy or publish flows

`user-invocable: false`

Hides the skill from the slash menu — useful for background knowledge skills

`allowed-tools`

Restricts which tools Claude can use during skill execution for tighter control

Simple mental model: Markdown body = *what* Claude should do. Frontmatter = *how* the skill behaves.

Arguments Make Skills Reusable

Using `$ARGUMENTS`

```
---
name: feature-breakdown
description: Turn a vague feature
  into a PRD
---

Convert this feature into a
full PRD:

$ARGUMENTS
```

The `$ARGUMENTS` placeholder accepts any input passed after the slash command, making the same skill work across dozens of different tasks.

Why This Changes Everything

1 One skill, many tasks

Reuse the same workflow across every product or marketing task without rewriting anything

2 Skills feel like mini tools

Pass different inputs and get consistent, structured outputs every time

3 Faster execution

No need to rebuild context — just invoke and pass the relevant input

Codepup

Advanced Pattern: Dynamic Context + Scripts

This is the shift from **prompt engineering** to **workflow engineering**. Skills can pull in live context and run scripts before Claude ever begins its response.

⚙️ Inject Command Output

Run a shell command and feed its output directly into the skill as live context — always current, never stale

📄 Preprocess Files

Convert PDFs and DOCs to markdown before Claude sees them — cleaner inputs mean more reliable, structured outputs

🖼️ Run Helper Scripts

Trigger image generation, formatting, or publishing scripts as part of the skill execution — not after it

In the Feature Breakdown Skill

PDF and DOC converters transform raw product briefs or customer feedback into clean markdown that Claude can reason over accurately.

In the Blog Writer Skill

An image generation script is bundled directly into the skill, producing visual assets as part of the same automated workflow.

Codepup

Best Practices for Building Skills

Keep skills narrow and practical

Small, sharp skills beat giant vague skills. A well-scoped skill for PRD generation outperforms a broad "PM assistant" skill every time.

Write a clear description

The description is how Claude decides when to auto-load the skill. A vague description leads to missed invocations or unwanted ones.

Include strong examples

Example outputs are the single highest-leverage thing you can add. They set the quality bar Claude will aim to match on every run.

Manually invoke risky actions

For any skill that deploys, publishes, commits, or sends — always set `disable-model-invocation: true`. You want to be the trigger.

A Simple Framework: Should This Be a Skill?

If you do something twice with the same structure, it is already a candidate. Use this framework to decide quickly.

Turn It Into a Skill If...

- You repeat it often across projects or sessions
- The structure and format of the output matters
- You need a consistent quality bar every time
- It benefits from examples or reference files
- It involves multiple steps, files, or scripts

Top PM & Founder Candidates

- PRD Generation
- Blog Creation
- Frontend Generation
- Social Media Posts
- Customer Email Drafting
- Research Summarisation

Codepup

Key Takeaways

> The real power is not just asking Claude better questions. It is packaging your best workflows so Claude can repeat them reliably.

Reduce Repetition

Define your workflow once in a `SKILL.md` file and invoke it with a single command — no more re-explaining context

Save Tokens

Reusable structure eliminates the overhead of re-describing workflows in every session, cutting costs significantly

Improve Consistency

Skills bundle examples, references, and rules so output quality stays high regardless of how you phrase the request

Unlock Real Automation

Bundle scripts, structured outputs, and preprocessing into skills to move beyond prompting into genuine workflow automation

Codepup